

**UNIVERSIDADE FEDERAL DE SANTA MARIA**  
**CENTRO DE TECNOLOGIA**  
**DEPARTAMENTO DE PROCESSAMENTO DE ENERGIA ELÉTRICA**



# **Polimorfismo**

---

Curso de Engenharia de Controle e Automação  
DPEE1090 - Programação orientada a objetos para automação

---

Prof. Renan Duarte

1º semestre de 2024

# Sumário

---

## Polimorfismo

- Definição de polimorfismo e sua importância na POO
- Polimorfismo estático: sobrecarga de métodos e operadores
- Polimorfismo dinâmico: substituição de métodos em tempo de execução

# Introdução

---

## Relembrando – Assinaturas de métodos

Refere-se ao conjunto de informações que identificam de forma única um método em uma classe ou interface. Inclui o nome do método e a lista de parâmetros (tipo e ordem).

A assinatura de método **não inclui** o tipo de retorno, os modificadores de acesso ou exceções lançadas pelo método.

Exemplos de métodos com assinaturas diferentes:

```
public int sum(double a, double b)
```

```
public int sum(int a, int b)
```

```
public int sum(int a, double b)
```

```
public int sum(double a, int b)
```

# Introdução

---

## Polimorfismo

**Poli → Muitas + Morfo → Formas**

Em POO, polimorfismo permite que um mesmo nome represente vários comportamentos diferentes.

### Exemplo conceitual:

Imagine um controle remoto universal que pode operar diferentes dispositivos eletrônicos como uma TV, um DVD player e um sistema de som. Cada dispositivo possui a sua própria implementação das funcionalidades básicas como ligar/desligar e ajustar o volume, mas todos eles podem ser controlados usando os mesmos botões no controle remoto universal.

# Introdução

---

## Tipos de polimorfismo

Existem alguns tipos diferentes de polimorfismo dependendo do contexto em que a mudança de comportamento ocorre. Neste curso, estudaremos os dois tipos principais, classificados de acordo com o momento em que a mudança de comportamento ocorre:

**Polimorfismo de sobrecarga (estático ou *Compile-time Polymorphism*)** → A resolução do método a ser chamado é feita durante a compilação do programa

**Polimorfismo de sobreposição/sobreescrita (dinâmico ou *Runtime Polymorphism*)** → A resolução do método a ser chamado é feita em tempo de execução

# Polimorfismo de sobrecarga

---

## Ocorre em tempo de compilação

**Definição:** Múltiplos métodos na mesma classe compartilham o mesmo nome, mas têm diferentes assinaturas

**Benefício:** Permite o uso de um único nome de método para diferentes operações baseadas nos parâmetros fornecidos

**Não envolve herança**

Exemplo:

```
class Calculadora
{
    public int Soma(int a, int b) { return a + b; }
    public double Soma(double a, double b) { return a + b; }
    public int Soma(int a, int b, int c) { return a + b + c; }
}
```

## Exemplo 1

Implementar uma classe de um sistema de monitoramento de sensores que possui um método chamado LerDados que opera com base nos seguintes critérios:

- Lê dados de um sensor de temperatura, recebendo como parâmetro um identificador do sensor (inteiro).
- Lê dados de um sensor de pressão, recebendo como parâmetro um identificador do sensor (inteiro) e a unidade de medida (string).
- Lê dados de um sensor de umidade, recebendo como parâmetro um identificador do sensor (inteiro), a unidade de medida (string) e a precisão desejada (double).

# Polimorfismo de sobreposição

---

## Ocorre em tempo de execução

**Definição:** Uma subclasse fornece uma implementação específica de um método que já é definido em sua superclasse

**Benefício:** Permite que a subclasse adapte ou estenda o comportamento de métodos da superclasse

**Permite tratar de forma homogênea objetos heterogêneos**

Exemplo:

```
class Animal
{
    virtual string Som() { return "O animal faz um som"; }
}
class Cachorro : Animal
{
    override string Som() { return "O cachorro faz um som"; }
}
```



## Exemplo 2

Crie uma classe base chamada Dispositivo com o método Operar. Crie as subclasses Sensor, Atuador e Controlador, todas herdando da classe Dispositivo. Implemente o método Operar em cada uma dessas subclasses para implementar comportamentos específicos:

- O método Operar na classe Sensor deve simular a leitura de dados
- O método Operar na classe Atuador deve simular a ativação de um atuador
- O método Operar na classe Controlador deve simular a execução de um algoritmo de controle

## Exemplo 3

Significado da frase “**Permite tratar de forma homogênea objetos heterogêneos**”

Qual o efeito de fazermos a declaração dos objetos na superclasse e a instanciação nas classes herdeiras?

```
Dispositivo MyAtuador = new Atuador();  
Dispositivo MySensor = new Sensor();  
Dispositivo MyControlador = new Controlador();
```

Qual comportamento ocorrerá quando chamarmos o método Operar de cada um destes objetos?

## Exemplo 3 – Continuação

Essa flexibilidade é um dos maiores benefícios que o polimorfismo traz a linguagens orientadas à objetos.

Pode-se por exemplo, gerenciar de uma maneira única e concisa, diferentes objetos de diferentes tipos de forma a obter um comportamento específico para cada objeto:

```
List<Dispositivo> Dispositivos = new List<Dispositivo>();  
Dispositivos.Add(new Atuador());  
Dispositivos.Add(new Sensor());  
Dispositivos.Add(new Controlador());  
  
foreach (var Disp in Dispositivos)  
    Disp.Operar();
```

# Revisão

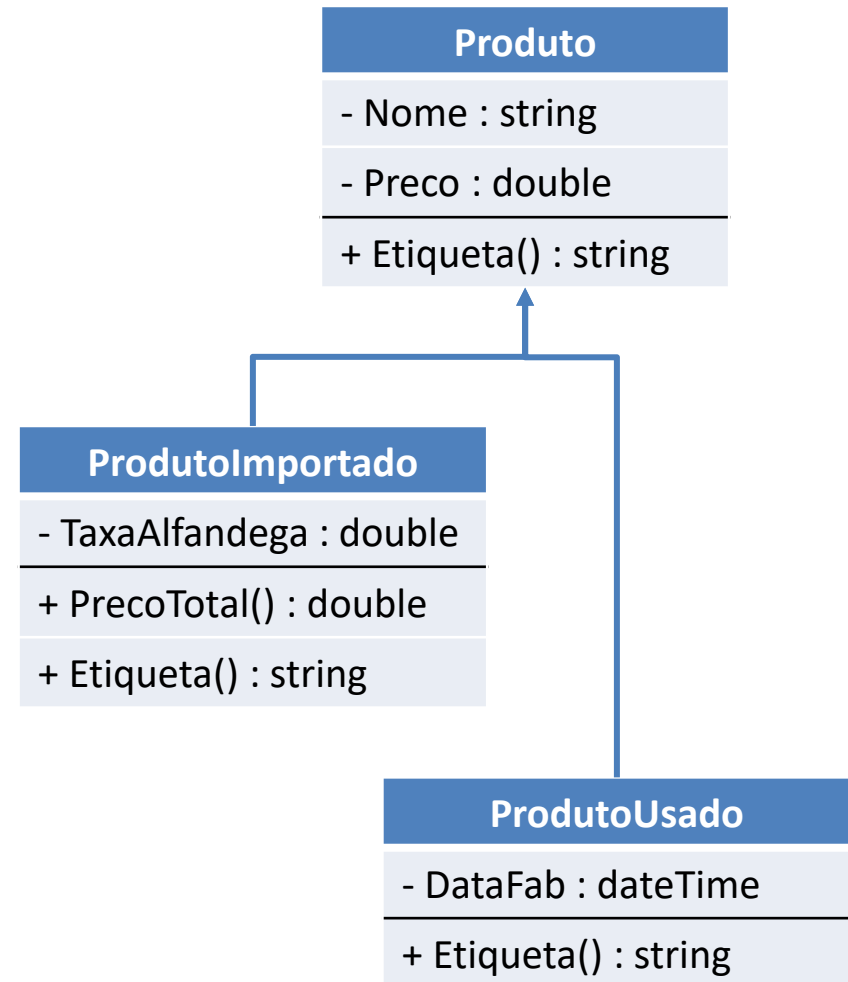
---

## Resumo

| Característica       | Estático              | Dinâmico                |
|----------------------|-----------------------|-------------------------|
| Momento de Resolução | Tempo de compilação   | Tempo de execução       |
| Baseado em           | Assinatura do método  | Herança e interfaces    |
| Flexibilidade        | Menor                 | Maior                   |
| Exemplo              | Sobrecarga de métodos | Sobrescrição de métodos |
| Ocorre em            | Mesma classe          | Classes diferentes      |

## Exercício 1

Fazer um programa para ler os dados de N produtos (N fornecido pelo usuário). Ao final, mostrar a etiqueta de preço de cada produto na mesma ordem em que foram digitados. Todo produto possui nome e preço. Produtos importados possuem uma taxa de alfândega, e produtos usados possuem data de fabricação. Estes dados específicos devem ser acrescentados na etiqueta de preço conforme exemplo (próximo slide). Para produtos importados, a taxa e alfândega deve ser acrescentada ao preço final do produto.



## Exercício 1 – Continuação

Digite o número de produtos: 3

Dados do produto #1:

Comum, usado ou importado (c/u/i)? i

Nome: Tablet

Preço: 260.00

Taxa de Alfândega: 20.00

Dados do produto #2:

Comum, usado ou importado (c/u/i)? c

Nome: Notebook

Preço: 1100

Dados do produto #3:

Comum, usado ou importado (c/u/i)? u

Nome: Iphone

Preço: 400

Data de fabricação (DD/MM/AAAA): 15/03/2017

Etiquetas de Preço:

Tablet – R\$ 280.00 (Taxa de Alfândega: R\$ 20.00)

Notebook – R\$ 1100.00

Iphone (usado) – R\$ 400.00 (Fabricação: 15/03/2017)

## Exercício 2

Você está desenvolvendo um sistema de monitoramento de sensores para uma planta industrial. Cada sensor pode ser de diferentes tipos, como temperatura, pressão, umidade, entre outros. Alguns sensores podem ser configurados como sensores de emergência, que exigem um tratamento especial no sistema de monitoramento.

Crie uma hierarquia de classes que represente diferentes tipos de sensores. Inclua uma classe base `Sensor` e subclasses como `SensorEmergencia`, `SensorTemperatura`, `SensorPressao`, etc. `Sensor` é abstrata e possui um método abstrato `Medir`.

Crie uma classe `SistemaMonitoramento` que representa o sistema global de monitoramento. Utilize listas ou arrays para armazenar diferentes sensores.

Implemente métodos na classe `SistemaMonitoramento` para realizar operações como iniciar o monitoramento, capturar dados dos sensores, aplicar lógica de controle de emergência, etc.

# Revisão

---

## Próxima aula

### Detecção e tratamento de erros

- Função try/catch
- Lançamento de exceções
- Exceções personalizadas



# Dúvidas?

---

[renan.duarte@gedre.ufsm.br](mailto:renan.duarte@gedre.ufsm.br)

GEDRE – Prédio 10 – CTLAB